# LAPORAN PRAKTIKUM MIKROPROSESOR DAN INTERFACE

# PERKALIAN BILANGAN OKTAL

Disusun untuk memenuhi tugas praktikum Mikroprosesor Dan Interface


Oleh

Gaitsha Azzahra Devitrya

231344012


Tanggal Percobaan: 16 Mei 2025

Tanggal Penyerahan: 23 Mei 2025

Instruktur: 1. Ferry Satria, BSEE., M.T.

2. Rahmawati Hasanah, S.ST., M.T.


**PROGRAM STUDI D4 TEKNIK TELEKOMUNIKASI**

**JURUSAN TEKNIK ELEKTRO**

**POLITEKNIK NEGERI BANDUNG**


**2025**

# DAFTAR ISI

**DAFTAR ISI**.....ii
**A. TUJUAN**.....2
**B. LANDASAN TEORI**.....2
**C. ALAT DAN BAHAN**.....4
**D. LANGKAH KERJA**.....5
**E. HASIL PERCOBAAN DAN ANALISIS DATA**.....6
**G. KESIMPULAN**.....20
**DAFTAR PUSTAKA**.....21

## A. TUJUAN

1. Mahasiswa dapat memahami konsep perkalian data bilangan oktal
2. Mahasiswa dapat membuat program pada EMU8086 untuk menyelesaikan operasi perkalian bilangan oktal.

## B. LANDASAN TEORI

### A. Definisi microprosesor

Mikroprosesor dikenal juga sebagai CPU (Central Processing Unit). CPU terdiri dari ALU, Control Unit, Register , Bus System, dan Clock untuk melakukan komputasi.

### B. Register

Register merupakan sebagian memori dari mikroprosesor yang dapat diakses dengan kecepatan yang sangat tinggi. Dalam melakukan pekerjaannya mikroprosesor selalu menggunakan register-register sebagai perantaranya, jadi register dapat diibaratkan sebagai kaki dan tangannya mikroprosesor.

Register merupakan sebagian memori dari mikroprosesor yang dapat diakses dengan kecepatan yang sangat tinggi. Dalam melakukan pekerjaannya mikroprosesor selalu menggunakan register-register sebagai perantaranya, jadi register dapat diibaratkan sebagai kaki dan tangannya mikroprosesor.

Terdapat dua jenis register yang dibedakan menurut besar Bit, yaitu Register 8-Bit dan Register 16-Bit.

Register 8-Bit hanya memiliki 1 bagian jenis register yaitu Register umum : AH, AL, BH, BL, CH, CL, DH, DL

Dan Register 16-Bit memiliki 5 bagian jenis register yang memiliki tugasnya masing-masing, yaitu :

1. Segmen Register.

Register yang termasuk dalam kelompok ini terdiri atas register CS (Code Segment), DS (Data Segment), ES (Extra Segment) dan SS (Stack Segment) yang masing-masingnya merupakan register 16 bit. Register-register dalam kelompok ini secara umum digunakan untuk menunjukkan alamat dari suatu segmen.

2. General Register (Register Umum)

Register yang termasuk dalam kelompok ini adalah register AX, BX, CX dan DX yang masing-masing terdiri atas 16 bit. Register- register 16 bit dari kelompok ini mempunyai suatu ciri khas, yaitu dapat dipisah menjadi 2 bagian dimana masing-masing bagian terdiri atas 8 bit, Akhiran H menunjukkan High sedangkan akhiran L menunjukkan Low.

3. Register Pointer

Register ini menunjukkan alamat sebuah data di lokasi memori, dipakai saat operasi perpindahan data(dari/ke memori) dan penunjukkan alamat suatu instruksi. Jenis-jenis register pointer yaitu BP (Base Pointer) dan IP (Instruction Pointer).

4. Register Index

Sama dengan pointer register, sering digunakan untuk menunjukkan alamat sebuah data di lokasi memori pada operasi string. Jenisnya ada SI (Source Index), DI (Destination Index).

5. Register Stack

Register Stack (SP) atau Stack Pointer adalah salah satu jenis register dalam arsitektur komputer yang digunakan untuk menunjuk alamat teratas dari stack dalam memori.

6. Register Flag [REG 16 BIT]

Register Flag berfungsi untuk menunjukkan status(keadaan) sesaat dari mikroprosesor. Bit-bit pada flag akan mengalami perubahan, tergantung proses yang berlangsung. Sesuai dengan namanya Flags(Bendera) register ini menunjukkan kondisi dari suatu keadaan< ya atau tidak >. Karena setiap keadaan dapat digunakan 1 bit saja, maka sesuai dengan jumlah bitnya, Flags register ini mampu memcatat sampai 16 keadaan.

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| - | - | - | - | OF | DF | IF | TF | SF | ZF | - | AF | - | PF | - | CF |

## C. Syntax/Perintah Mikroprosesor 8086

1. ADC: Penjumlahan dengan Carry
2. ADD: Penjumlahan
3. CMP : Membandingkan
   Memiliki Algoritma: Operand 1 – Operand 2
   Hasil tidak disimpan dimana-mana, hanya saja mempengaruhi Flag (OF, SF,ZF, AF, PF, CF) berdasarkan hasil.
4. MOV : Instruksi MOV tidak bisa dilaksanakan jika:
   - Memberikan nilai ke Register CS dan Register IP
   - Tidak bisa Menyalin nilai dari satu segmen ke segmen Register yang lain (Seharusnya menggunakan Register umum terlebih dahulu)
   - Menyalin segera data ke segmen register ( Seharusnya menggunakan Register umum terlebih dahulu)
5. INC : Menaikkan
   Memiliki Algoritma : Operand = Operand + 1

6. JNZ(Jump if Not Zero) = untuk melakukan lompatan ke alamat tertentu jika hasil operasi sebelumnya tidak nol.
7. DUP : untuk pendeklarasian variabel di segmen .DATA untuk mengalokasikan memori dalam jumlah tertentu dengan nilai awal yang sama.
8. DAT bukan instruksi bawaan Assembly, yang digunakan adalah DB (Define Byte), DW (Define Word), atau DD (Define Double Word) untuk mendefinisikan data.
   - DB (Define Byte) → Menyimpan 1 byte data
   - DW (Define Word) → Menyimpan 2 byte data
   - DD (Define Double Word) → Menyimpan 4 byte data

**D. Perintah DAA (Decimal Adjust after Addition)**

Perintah DAA (Decimal Adjust after Addition) digunakan untuk menyesuaikan hasil penjumlahan agar tetap dalam format BCD (Binary-Coded Decimal) setelah operasi penambahan. Instruksi ini bekerja dengan memeriksa nilai di register AL setelah operasi penjumlahan biasa menggunakan ADD atau ADC**.** Jika bagian rendah dari register AL (4 bit paling rendah) lebih besar dari 9 atau jika AF (Auxiliary Carry Flag) diatur, maka mikroprosesor akan menambahkan 6 (06H) ke bagian rendah AL untuk memastikan hasil tetap dalam format BCD. Demikian pula, jika bagian tinggi AL (4 bit paling atas) lebih besar dari 9 atau jika CF (Carry Flag) diatur, maka mikroprosesor akan menambahkan 60H ke AL dan mengatur CF sebagai tanda bahwa ada carry ke digit berikutnya. Perintah DAA umumnya digunakan dalam aplikasi yang menangani angka desimal.

**E. Subrutin pada mikroprosesor**

Subrutin pada mikroprosesor emu8086 adalah bagian dari program yang dirancang untuk menjalankan tugas tertentu dan dapat dipanggil dari bagian lain dalam program utama. Subrutin digunakan untuk mengurangi redundansi kode, meningkatkan keterbacaan, serta mempermudah pemeliharaan program. Dalam assembly 8086, subrutin dideklarasikan sebagai procedure (PROC) dan diakhiri dengan RET untuk kembali ke pemanggilnya.

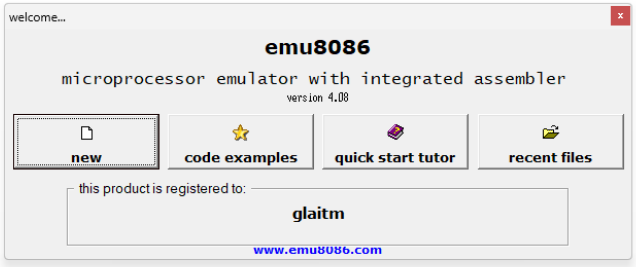
Ketika sebuah subrutin dipanggil menggunakan instruksi CALL**,** alamat instruksi berikutnya akan disimpan di stack agar setelah eksekusi subrutin selesai, program dapat kembali ke titik pemanggilan dengan menggunakan instruksi RET. Proses ini memungkinkan penggunaan kembali kode tanpa perlu menuliskan instruksi yang sama berulang kali.

## C. ALAT DAN BAHAN

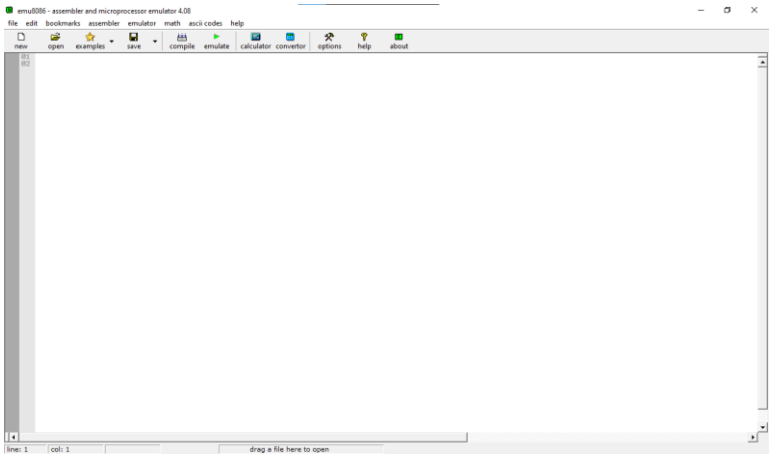
1. Laptop/PC
2. Software EMU 8086

## D. LANGKAH KERJA

1. Instal Software emu8086 pada laptop dan buka aplikasinya
2. Setelah membuka aplikasi maka akan muncul tampilan *new* pada monitor

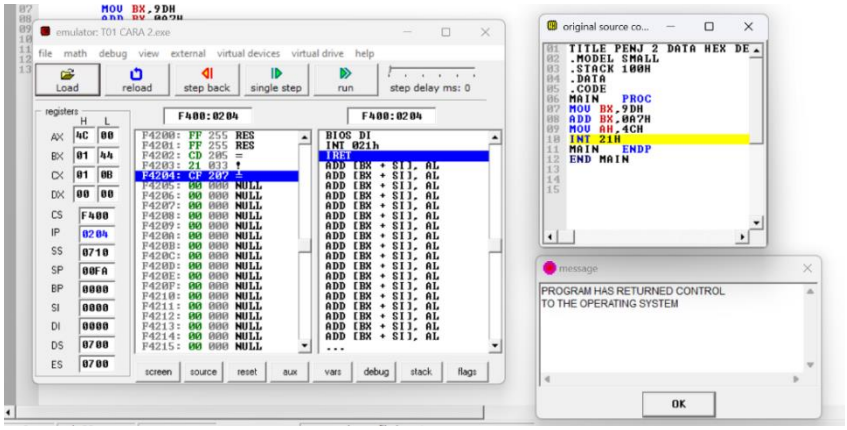



3. Maka setelah itu akan tampil worksheet seperti gambar dibawah ini :



Worksheet ini digunakan untuk menuliskan kode.

4. Untuk memeriksa kode yang berada di worksheet apakah program sudah benar atau belum, klik “compile”
5. Jika sudah disave maka jika program sudah benar akan ada tulisan “run”.
6. Jika sudah di ”run” maka akan muncul hasilnya pada monitor



## E. HASIL PERCOBAAN DAN ANALISIS DATA

Pada praktikum ini, menggunakan aplikasi emu8086 dimana akan menghitung perkalian bilangan oktal. Perkalian dilakukan dengan menggunakan metode penjumlahan berulang, pada programnya dapat menggunakan instruksi aritmetika seperti ADD atau ADC. Namun, hasil penjumlahan bisa keluar dari format oktal, sehingga perlu dikoreksi menggunakan instruksi CALL SDAA (Decimal Adjust after Addition) valid dalam sistem oktal. Dalam implementasi, angka-angka BCD biasanya disimpan di memori, kemudian diakses menggunakan indeks (SI atau DI) dalam sebuah loop untuk dijumlahkan satu per satu.

Kemudian pada praktikum kali ini juga, terdapat bagian tambahan pada program yang disebut dengan subrutin, bagian ini digunakan pada program jika hasil penjumlahan yang akan ditampilkan dalam bentuk bilangan oktal masih belum benar. Subrutin ini mengecek setiap digit satuan dan puluhan, lalu menambahkan 8 jika diperlukan untuk mempertahankan format oktal.

Kode program subrutin diperlukan pada praktikum ini, karena pada praktikum ini hampir semuanya merupakan penjumlahan yang akan ditampilkan dalam format oktal.Kode subrutin yang digunakan pada praktikum kali ini adalah sebagai berikut :

```
SDAA PROC   ; SUBRUTIN SDAA
    PUSH BX
    PUSH CX
    MOV BH,AH
    LAHF
    MOV BL,AH
    MOV CH,AL
    MOV CL,AL
    AND CL, 0FH; CL=SATUAN
    ;==============
    TEST AH, 10H
    JZ L22
    ADD AL, 8
    JNC L22
    ADD AL, 80H
    OR BL,1
L22: TEST AH,1
    JZ L33
    ADD AL,80H
    OR BL,1
L33: MOV CH,AL
    AND CH,0FH
    CMP CH,7
    JBE L44
    ADD AL,8
    JNC L44
    ADD AL,80H
    OR BL,1
L44: MOV CH,AL
    AND CH,0F0H
    CMP CH,70H
    JBE L55
    ADD AL,80H
    OR BL,1
L55: MOV AH,BL
    SAHF
    POP CX
    POP BX
    MOV AH,BH
    RET
SDAA ENDP
```

Pada praktikum kali ini terdapat 5 soal yang berhasil dikerjakan yang dapat dianalisis sebagai berikut :

1. Buat program untuk mengalikan data Oktal 2 digit yang ada pada reg BL dengan data Oktal 2 digit yang ada pada reg CH. Simpan Hasil perkalian pada reg CL:DX.

Program :

```
TITLE NO1 (OKTAL)
.MODEL SMALL
.STACK 100H
.DATA
.CODE
MAIN PROC
     ADD BL,0
     JZ AKHIR
     ADD CH,0
     JZ AKHIR
     MOV DX,0

SINI:     MOV AL,DL
     ADD AL,BL
     CALL SDAA
     MOV DL,AL

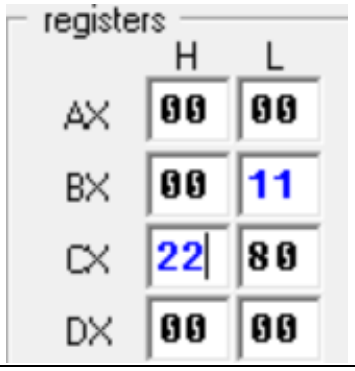
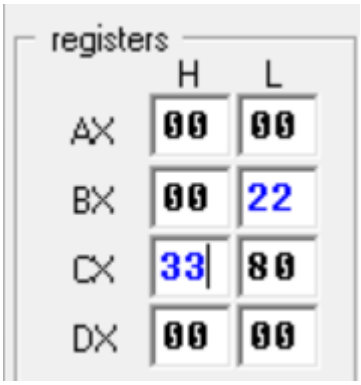
     MOV AL,DH
     ADC AL,0
     CALL SDAA
     MOV DH,AL

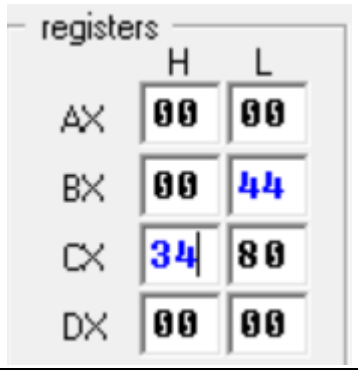
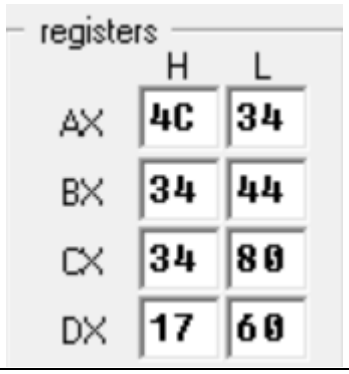
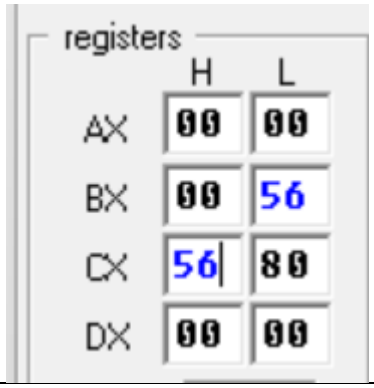
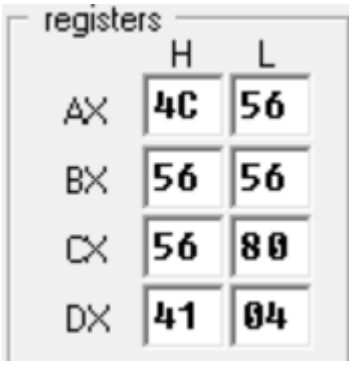
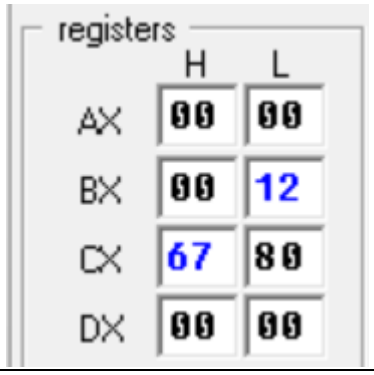
     MOV AL,BH
     ADD AL,1
     CALL SDAA
     MOV BH,AL
     CMP BH,CH
     JNZ SINI

AKHIR:    MOV AH,4CH
     INT 21H
MAIN ENDP
```

```
AKHIR:    MOV AH,4CH
     INT 21H
MAIN ENDP
SDAA PROC    ; SUBRUTIN SDAA
     PUSH BX
     PUSH CX
     MOV BH,AH
     LAHF
     MOV BL,AH
     MOV CH,AL
     MOV CL,AL
     AND CL, 0FH; CL=SATUAN
     ;==============
     TEST AH, 10H
     JZ L22
     ADD AL, 8
     JNC L22
     ADD AL, 80H
     OR BL,1
L22: TEST AH,1
     JZ L33
     ADD AL,80H
     OR BL,1
L33: MOV CH,AL
     AND CH,0FH
     CMP CH,7
     JBE L44
     ADD AL,8
     JNC L44
     ADD AL,80H
     OR BL,1
L44: MOV CH,AL
     AND CH,0F0H
     CMP CH,70H
     JBE L55
     ADD AL,80H
     OR BL,1
L55: MOV AH,BL
     SAHF
     POP CX
     POP BX
     MOV AH,BH
     RET
SDAA ENDP
END MAIN
```

Hasil program:

| Data Percobaan | Hasil Percobaan |
|---|---|
| 11 * 22 = 242  |  |
| 22 * 33 = 746  |  |

| | |
|---|---|
| 44 * 34 = 1760<br>  |  |
| 56 * 56 = 4104<br>  |  |
| 12 * 67 = 1046<br>  |  |

Analisis Data :

Pada percobaan pertama yaitu membuat program Assembly pada EMU8086 merupakan implementasi dari perkalian dua bilangan dalam bentuk BCD (Binary Coded Decimal) 8-bit atau bilangan oktal, di mana hasilnya disimpan dalam format 16-bit. Program ini menggunakan subrutin SDAA untuk menangani operasi penambahan dalam format BCD.

Program dimulai dengan membaca dua buah nilai BCD yang disimpan dalam register BL dan CH. Nilai awal dari DX diset ke nol sebagai penampung hasil. Proses utama dilakukan dalam label SINI, di mana dilakukan penambahan nilai AL dengan BL, kemudian dipanggil subrutin SDAA untuk mengatur hasil penambahan agar tetap dalam format BCD yang valid. Nilai dari DL (bagian bawah DX) kemudian diisi dengan nilai AL.

Berikutnya, AL diisi dengan nilai DH dan dilakukan proses ADC (Add with Carry) dengan nol, kemudian SDAA kembali dipanggil dan nilai AL disimpan ke DH, yang artinya ini menyimpan bagian atas dari hasil (karena hasil perkalian bisa mencapai 16-bit). Perulangan dilakukan berdasarkan perbandingan antara BH dan CH (yang mewakili angka yang dikalikan), dan jika belum sama, maka akan mengulangi proses penambahan. Subrutin SDAA menangani penyesuaian nilai BCD hasil penjumlahan. Program perkalian menggunakan metode penjumlahan berulang dalam format BCD. Dengan menggunakan loop dan penambahan berulang, nilai hasil dikonstruksi secara bertahap dan dikoreksi untuk memastikan setiap digit BCD tetap valid.

Hasil percobaan menunjukkan bahwa ketika diberikan masukan 11 dan 22, maka hasil akhir dalam register DX adalah 00F2 (atau 242 dalam oktal), yang sesuai dengan 11 × 22. Begitu juga ketika diberikan 22 dan 33, maka hasil akhirnya adalah 02EA (atau 746), sesuai dengan 22 × 33. Ini membuktikan bahwa logika program bekerja dengan benar dan subrutin juga berhasil mengoreksi hasil setiap kali diperlukan.

2. Buat Program untuk mengalikan data oktal 4 digit yang ada pada register BX dengan data oktal 4 digigt yang ada pada register CX. Simpan hasil perkalian pada register SI:DX.

Program :

```
TITLE NO2 (OKTAL)
.MODEL SMALL
.STACK 100H
.DATA
DAT DB  0,0,0,0,0,0
.CODE
MAIN PROC
     MOV AX, @DATA
     MOV DS, AX
     ADD BX,0
     JZ AKHIR
     ADD CX,0
     JZ AKHIR
     MOV SI,0
     MOV DX,0

SINI:     MOV AL,[DAT+2]
     ADD AL,BL
     CALL SDAA
     MOV [DAT+2],AL

     MOV AL,[DAT+1]
     ADC AL,BH
     CALL SDAA
     MOV [DAT+1],AL

     MOV AL, [DAT]
     ADC AL,0
     CALL SDAA
     MOV [DAT],AL

     MOV AL,[DAT+5]
     ADD AL,1
     CALL SDAA
     MOV [DAT+5],AL
     CMP [DAT+5],CL
     JNZ SINI
```

```
SANA:     MOV AL,DL
     ADD AL,BL
     CALL SDAA
     MOV DL,AL

     MOV AL,DH
     ADC AL,BH
     CALL SDAA
     MOV DH,AL

     MOV AL,[DAT+3]
     ADC AL,0
     CALL SDAA
     MOV [DAT+3],AL

     MOV AL,[DAT+6]
     ADD AL,1
     CALL SDAA
     MOV [DAT+6],AL
     CMP [DAT+6],CH
     JNZ SANA

     MOV AL, [DAT+1]
     ADD AL, DL
     CALL SDAA
     MOV [DAT+1],AL

     MOV AL, [DAT]
     ADC AL, DH
     CALL SDAA
     MOV [DAT],AL

     MOV AL, [DAT+3]
     ADC AL, 0
     CALL SDAA
     MOV [DAT+3],AL

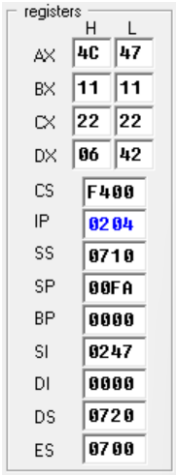
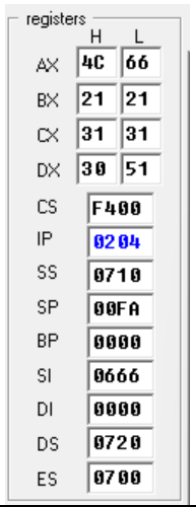
     MOV DH, [DAT+1]
     MOV DL, [DAT+2]

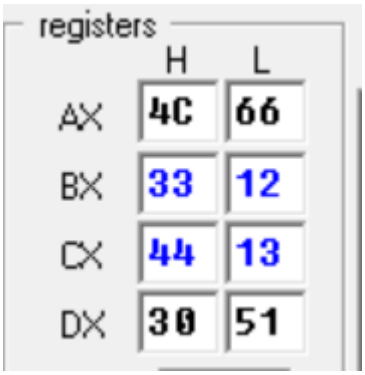
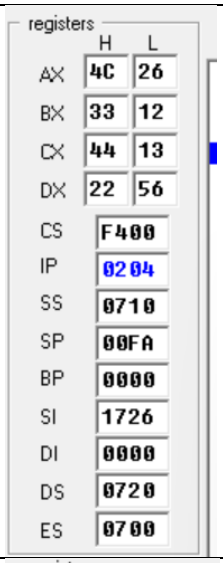
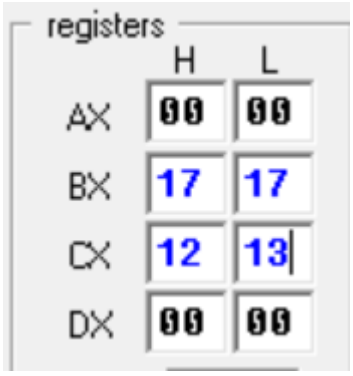
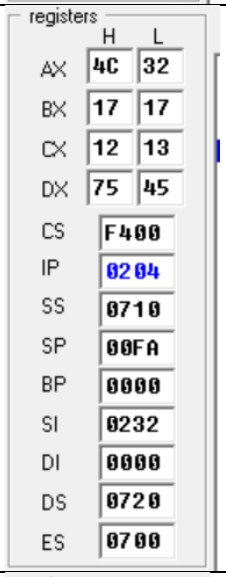
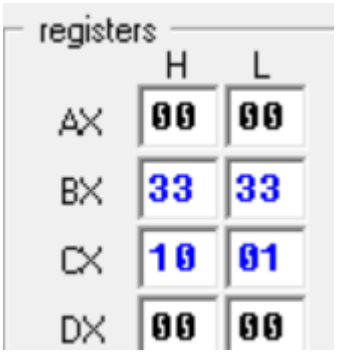
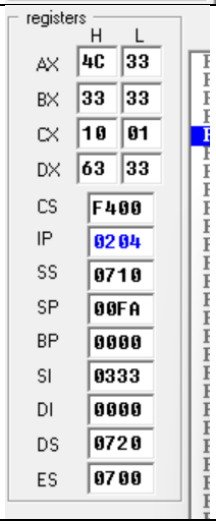
     MOV AL, [DAT]
     MOV AH, [DAT+3]
     MOV SI, AX

AKHIR:    MOV AH,4CH
     INT 21H
```

```
MAIN ENDP
SDAA PROC    ; SUBRUTIN SDAA
     PUSH BX
     PUSH CX
     MOV BH,AH
     LAHF
     MOV BL,AH
     MOV CH,AL
     MOV CL,AL
     AND CL, 0FH; CL=SATUAN
     ;==============
     TEST AH, 10H
     JZ L22
     ADD AL, 8
     JNC L22
     ADD AL, 80H
     OR BL,1
L22: TEST AH,1
     JZ L33
     ADD AL,80H
     OR BL,1
L33: MOV CH,AL
     AND CH,0FH
     CMP CH,7
     JBE L44
     ADD AL,8
     JNC L44
     ADD AL,80H
     OR BL,1
L44: MOV CH,AL
     AND CH,0F0H
     CMP CH,70H
     JBE L55
     ADD AL,80H
     OR BL,1
L55: MOV AH,BL
     SAHF
     POP CX
     POP BX
     MOV AH,BH
     RET
SDAA ENDP
END MAIN
```

Hasil program:

| Data Percobaan | Hasil Percobaan |
|---|---|
| 1111* 2222 == 2470642   |   |
| 2121 * 3131 = 6663051   |   |

| | |
|---|---|
| 3312 * 4413 = 17262256<br>  <br> |  <br> |
| 1717 * 1213 = 2327545<br>  <br> |  <br> |
| 3333 * 1000 = 3336333<br>  <br> |  <br> |

Analisis Data :

Pada percobaan kedua yaitu membuat program Assembly pada EMU8086 merupakan implementasi dari perkalian dua bilangan dalam bentuk BCD (Binary Coded Decimal) 8-bit atau bilangan oktal. Program ini menggunakan subrutin SDAA untuk menangani operasi penambahan dalam format BCD. Perbedaannya, program ini menggunakan array DAT sebagai tempat penyimpanan nilai operand, hasil sementara, dan hasil akhir, bukan sekadar register. Program ini merupakan perkalian bilangan oktal 4 digit x 4 digit.

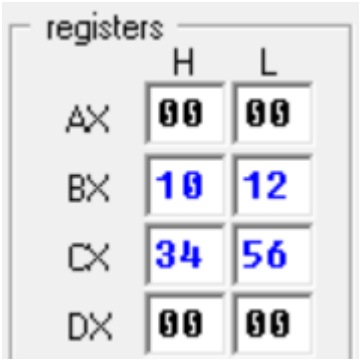
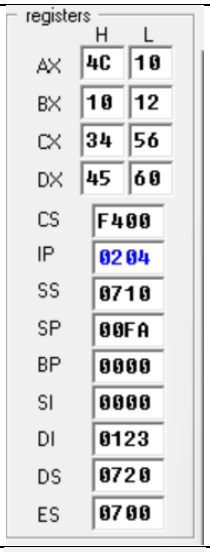
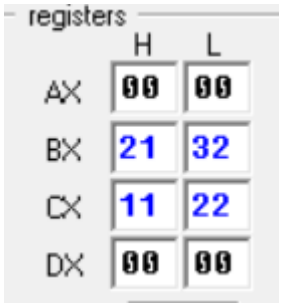
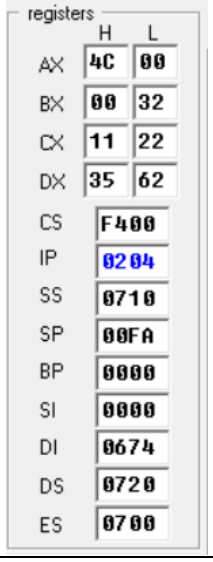
Pada bagian awal, AX diisi dengan segment alamat dari DAT, dan dipindahkan ke DS. Register BX, CX, dan DX diinisialisasi untuk digunakan sebagai penghitung loop dan penyimpan hasil sementara. Dua bilangan yang akan dikalikan diambil dari DAT[2] dan DAT[5]. Label SINI: adalah bagian utama dari loop yang akan mengalikan dua angka dengan cara menambahkan angka pertama (dari DAT[2]) sebanyak nilai angka kedua (dari DAT[5]) secara berulang. Penjumlahan dilakukan melalui pemanggilan subrutin SDAA, yang bertugas untuk mengatur hasil penjumlahan agar tetap valid dalam format BCD. Hasil penjumlahan secara bertahap disimpan di DAT[0], DAT[1], DAT[3], dan DAT[6], bergantung pada bit hasil dan carry. Label SANA: adalah bagian tambahan yang menangani carry (nilai sisa) ke byte yang lebih tinggi dalam array DAT jika hasil overflow terjadi. Setelah proses pengulangan selesai (saat nilai penghitung di CL dan DAT[5] sama), eksekusi diarahkan ke AKHIR, yang menggunakan INT 21h untuk keluar dari program. Dan hasil akhir program disimpan pada register SI : DX.

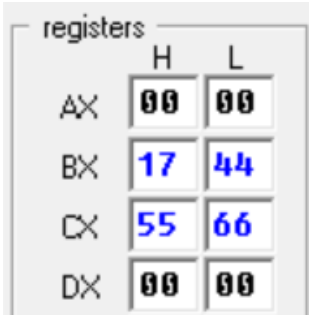
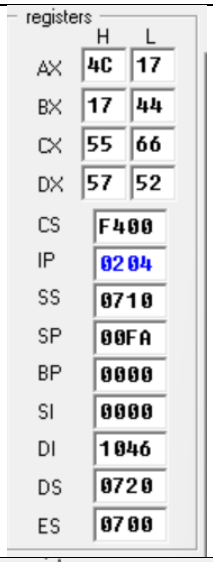
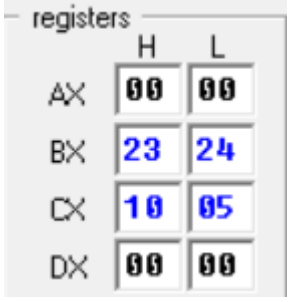
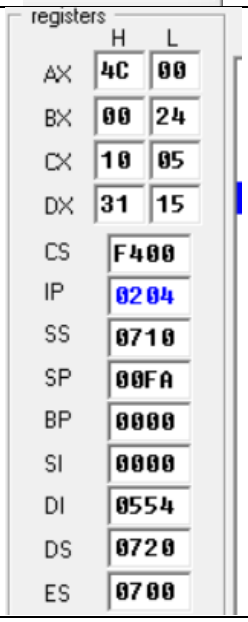
3. Buat program untuk mengalikan data oktal 6 digit yang ada pada register BL:CX dengan data 2 digit yang ada pada register BH. Simpan hasil perkalian pada regsiter DI : DX

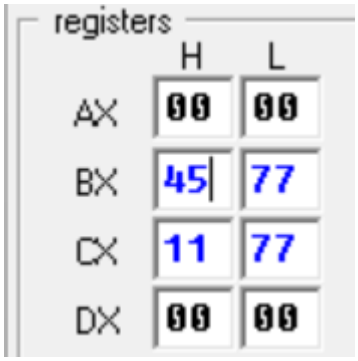
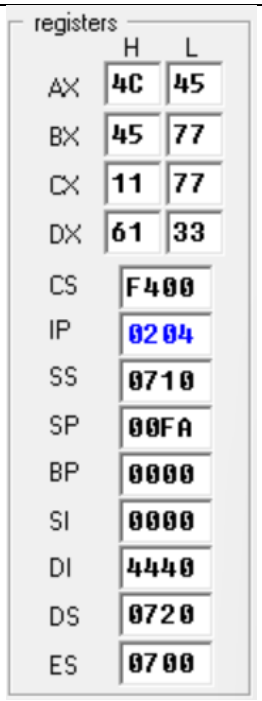
Program :

```
01 TITLE PERKALIAN NO3 (OKTAL)
02 .MODEL SMALL
03 .STACK 100H
04 .DATA
05 DAT DB 0,0,0,
06 .CODE
07 MAIN PROC
08       MOV AX, @DATA
09       MOV DS, AX
10       MOV DI, 0
11       MOV DX, 0
12       CMP BH,0
13       JZ SANA
14       CMP CX, 0
15       JZ SANA
16       CMP BL, 0
17       JZ SANA
18
19 SINI:MOV AL, DL
20       ADD AL, CL
21       CALL SDAA
22       MOV DL, AL
23
24       MOV AL, DH
25       ADC AL, CH
26       CALL SDAA
27       MOV DH, AL
28
29       MOV AL, [DAT]
30       ADC AL, BL
31       CALL SDAA
32       MOV [DAT], AL
33
34       MOV AL, [DAT+1]
35       ADC AL, 0
36       CALL SDAA
37       MOV [DAT+1], AL
38
39       MOV AL, [DAT]
40       MOV AH, [DAT+1]
41       MOV DI, AX
42
43       MOV AL, [DAT+2]
44       ADD AL, 1
45       CALL SDAA
46       MOV [DAT+2], AL
47       CMP [DAT+2],BH
48       JNZ SINI
49 SANA:MOV AH, 4CH
50       INT 21H
51 MAIN ENDP
52 SDAA PROC    ; SUBRUTIN SDAA
53       PUSH BX
54       PUSH CX
55       MOV BH,AH
56       LAHF
57       MOV BL,AH
58       MOV CH,AL
59       MOV CL,AL
60       AND CL, 0FH; CL=SATUAN
61       ;===============
62       TEST AH, 10H
63       JZ L22
64       ADD AL, 8
65       JNC L22
66       ADD AL, 80H
67       OR BL,1
68 L22: TEST AH,1
69       JZ L33
70       ADD AL,80H
71       OR BL,1
72 L33: MOV CH,AL
73       AND CH,0FH
74       CMP CH,7
75       JBE L44
76       ADD AL,8
77       JNC L44
78       ADD AL,80H
79       OR BL,1
80 L44: MOV CH,AL
81       AND CH,0F0H
82       CMP CH,70H
83       JBE L55
84       ADD AL,80H
85       OR BL,1
86 L55: MOV AH,BL
87       SAHF
88       POP CX
89       POP BX
90       MOV AH,BH
91       RET
92 SDAA ENDP
93 END MAIN
```

Hasil program:

| Data Percobaan | Hasil Percobaan |
|---|---|
| 123456 * 10 = 1234560  |  |
| 321122 * 21 = 6743562  |  |

| | |
|---|---|
| 445566 * 17 = 10465752<br>  <br> |  <br> |
| 241005 * 23 = 5.543.115<br>  <br> |  <br> |

| | |
|---|---|
| 771177 * 45 = 44406133<br>  <br> |  <br> |

Analisis Data :

Pada percobaan ketiga yaitu membuat program Assembly pada EMU8086 merupakan implementasi dari perkalian dua bilangan dalam bentuk BCD (Binary Coded Decimal) 8-bit atau bilangan oktal. Program ini menggunakan subrutin SDAA untuk menangani operasi penambahan dalam format BCD. Perbedaannya, program ini menggunakan array DAT sebagai tempat penyimpanan nilai operand, hasil sementara, dan hasil akhir, bukan sekadar register. Program ini merupakan perkalian bilangan oktal 6 digit x 2 digit.

Bilangan BCD 6 digit yang dimasukkan adalah 123456, dan dikalikan dengan angka 10, menghasilkan output 1234560, seperti terlihat pada hasil register setelah eksekusi. Pada bagian awal, program menginisialisasi segment data dan register yang akan digunakan, kemudian melakukan pengecekan nilai input untuk memastikan data valid. Selanjutnya, proses perkalian dilakukan dengan memanfaatkan register-register seperti AL, DL, DH, dan menggunakan subrutin bernama SDAA untuk menangani koreksi BCD. Instruksi seperti DAA, AND, OR, dan TEST digunakan untuk memastikan hasil matematikanya tetap akurat secara oktal. Hasil akhirnya ditampilkan dalam bentuk register DI : DX, menunjukkan bahwa proses perkalian berhasil dengan benar.

4.Buat program untuk mengalikan data oktal 4 digit yang ada pada register BX dengan data oktal 6 digit yang ada pada register CH:DX. Simpan hasil perkalian pada data area[10 digit]

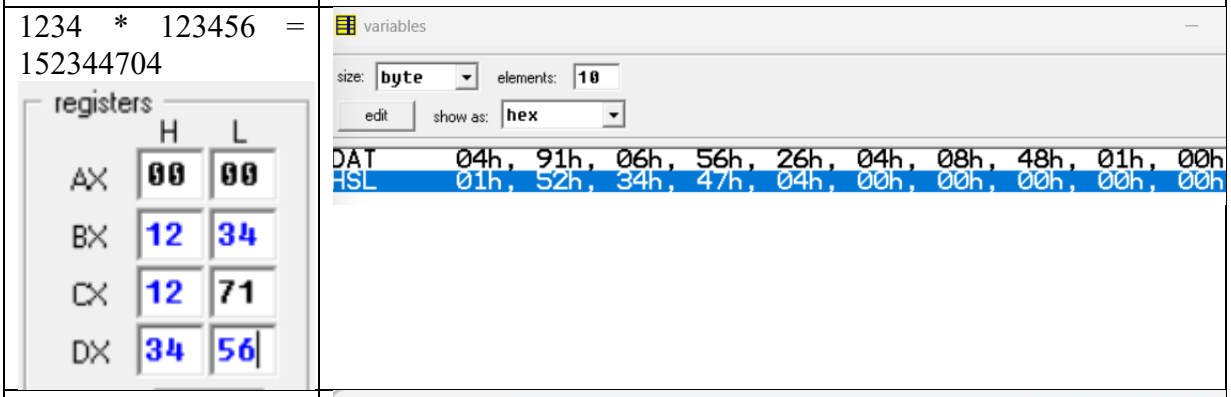
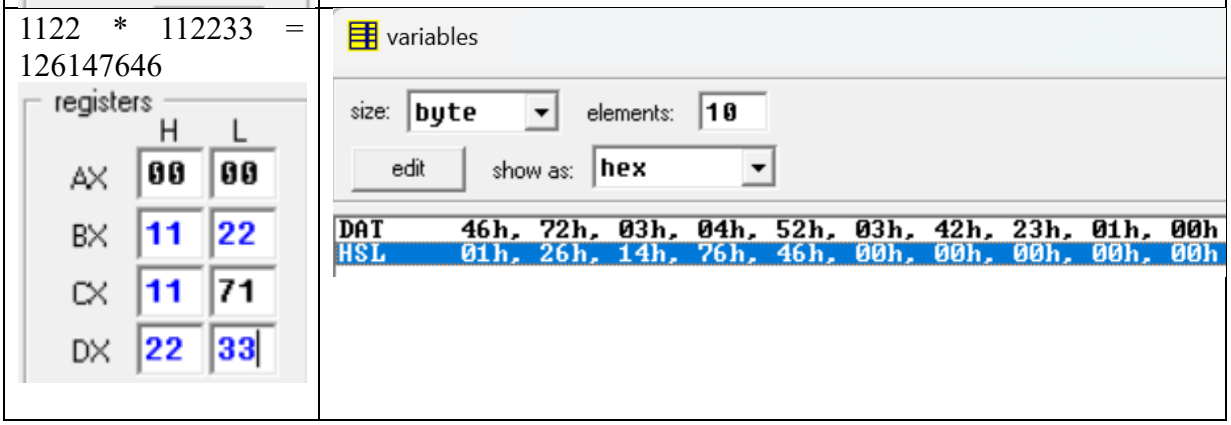
Program :

```
001 TITLE N04 (OKTAL)
002 .MODEL SMALL
003 .STACK 100H
004 .DATA
005 DAT DB 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0
006 HSL DB 0,0,0,0,0
007 .CODE
008 MAIN PROC
009       MOV AX, @DATA
010       MOV DS, AX
011       CMP CX, 0
012       JZ SANA
013       CMP BX, 0
014       JZ SANA
015
016   SINI:   MOV AL, [DAT]
017           ADD AL, BL
018           CALL SDAA
019           MOV [DAT], AL
020           MOV AL, [DAT+1]
021           ADC AL, BH
022           CALL SDAA
023           MOV [DAT+1], AL
024           MOV AL, [DAT+2]
025           ADC AL, 0
026           CALL SDAA
027           MOV [DAT+2], AL
028           MOV AL, [DAT+11]
029           ADD AL, 1
030           CALL SDAA
031           MOV [DAT+11], AL
032           CMP [DAT+11],DL
033           JNZ SINI
034
035     HAI:MOV AL, [DAT+3]
036           ADD AL, BL
037           CALL SDAA
038           MOV [DAT+3], AL
039           MOV AL, [DAT+4]
040           ADC AL, BH
041           CALL SDAA
042           MOV [DAT+4], AL
043           MOV AL, [DAT+5]
044           ADC AL, 0
045           CALL SDAA
046           MOV [DAT+5], AL
```

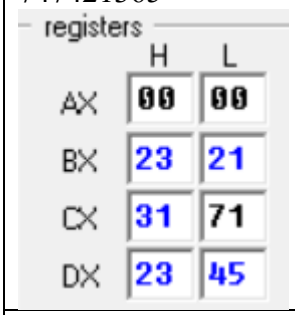
```
047           MOV AL, [DAT+12]
048           ADD AL, 1
049           CALL SDAA
050           MOV [DAT+12], AL
051           CMP [DAT+12],DH
052           JNZ HAI
053
054     HOI:MOV AL, [DAT+6]
055           ADD AL, BL
056           CALL SDAA
057           MOV [DAT+6], AL
058           MOV AL, [DAT+7]
059           ADC AL, BH
060           CALL SDAA
061           MOV [DAT+7], AL
062           MOV AL, [DAT+8]
063           ADC AL, 0
064           CALL SDAA
065           MOV [DAT+8], AL
066           MOV AL, [DAT+13]
067           ADD AL, 1
068           CALL SDAA
069           MOV [DAT+13], AL
070           CMP [DAT+13],CH
071           JNZ HOI
072           MOV AL, [DAT]
073           MOV [HSL+4], AL
074           MOV AL, [DAT+1]
075           ADC AL, [DAT+3]
076           CALL SDAA
077           MOV [HSL+3], AL
078           MOV AL, [DAT+2]
079           ADC AL, [DAT+4]
080           CALL SDAA
081           MOV [DAT+4], AL
082           MOV AL, [DAT+9]
083           ADC AL, 0
084           CALL SDAA
085           MOV [DAT+9], AL
086           MOV AL, [DAT+4]
087           ADD AL, [DAT+6]
088           CALL SDAA
089           MOV [HSL+2], AL
090           MOV AL, [DAT+9]
091           ADC AL, [DAT+5]
092           CALL SDAA
```

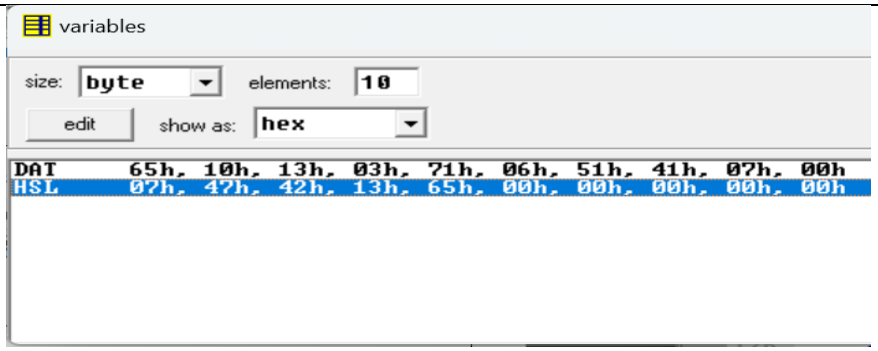
```
093           MOV [DAT+5], AL
094           MOV AL, [DAT+10]
095           ADC AL, 0
096           CALL SDAA
097           MOV [DAT+10], AL
098           MOV AL, [DAT+5]
099           ADD AL, [DAT+7]
100           CALL SDAA
101           MOV [HSL+1], AL
102           MOV AL, [DAT+8]
103           ADC AL, [DAT+10]
104           CALL SDAA
105           MOV [HSL], AL
106 SANA:MOV AH, 4CH
107       INT 21H
108 MAIN ENDP
109 SDAA PROC    ; SUBRUTIN SDAA
```

Hasil program:

| Data Percobaan | Hasil Percobaan |
|---|---|
| 1234 * 123456 = 152344704 |  |
| 1122 * 112233 = 126147646 |  |

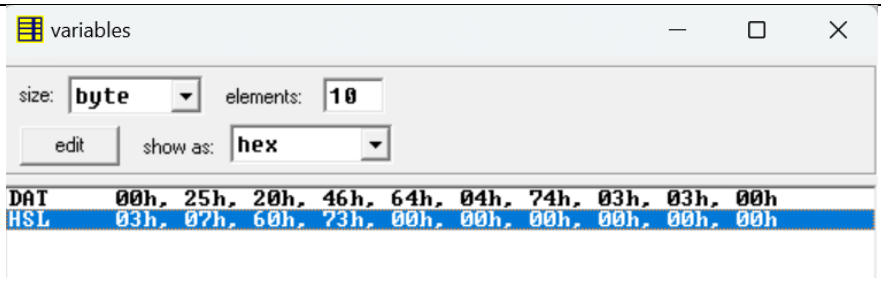
2321 * 312345 = 747421365



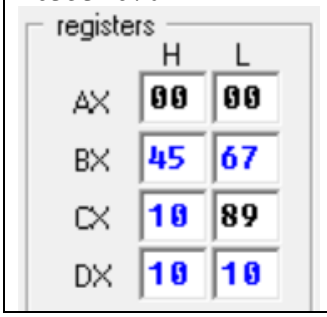



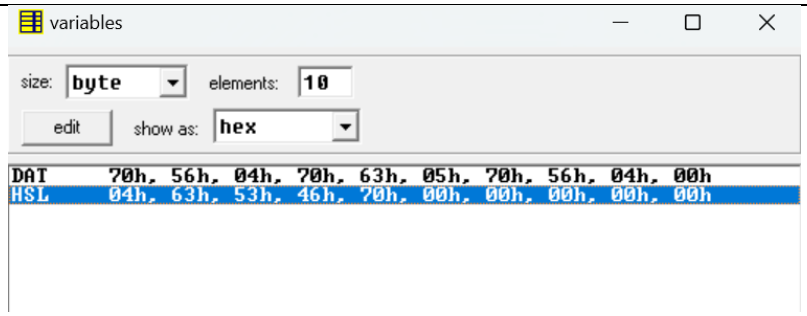

2025 * 141516 = 307236546




4567 * 101010= 463534670




Analisis Data :

Program soal ke -4 ini adalah program assembly pada EMU8086 yang implementasi dari perkalian bilangan dalam oktal (basis 8). Pada bagian awal program, data bilangan yang akan dikalikan didefinisikan dalam array DAT dan hasilnya disimpan pada array HSL. Program memulai dengan inisialisasi segment register DS, kemudian melakukan pemeriksaan awal terhadap nilai register CX dan BX untuk memastikan tidak bernilai nol sebelum melanjutkan ke bagian utama.

Perhitungan ada pada tiga struktur loop : SINI, HAI, dan HOI. Masing-masing bagian menangani digit-digit tertentu dari data masukan DAT, mengolahnya melalui penambahan dan carry menggunakan instruksi ADD, ADC, dan hasilnya diperbaiki menggunakan subrutin SDAA. Subrutin SDAA berfungsi sebagai koreksi untuk memastikan setiap digit hasil tetap dalam format oktal yang benar. Koreksi dilakukan dengan memeriksa nilai biner dan menyesuaikannya agar tetap valid dalam sistem oktal. Proses ini dilakukan dengan manipulasi bit dan kondisi logika yang kompleks menggunakan instruksi TEST, JNC, OR, dan AND.

Akhir dari program melakukan penggabungan dan penyimpanan hasil ke dalam array HSL, dan kemudian menghentikan program menggunakan INT 21H. Program ini menunjukkan bagaimana perkalian bilangan oktal dilakukan tanpa menggunakan instruksi perkalian langsung dari prosesor, melainkan melalui simulasi aritmatika dasar yang dikendalikan sepenuhnya di tingkat register dan memori.

5.Buat program untuk mengalikan data desimal 6 digit yang ada pada register CL : BX dengan data desimal 6 digit yang ada pada register CH:DX. Simpan hasil perkalian pada data area[12 digit]

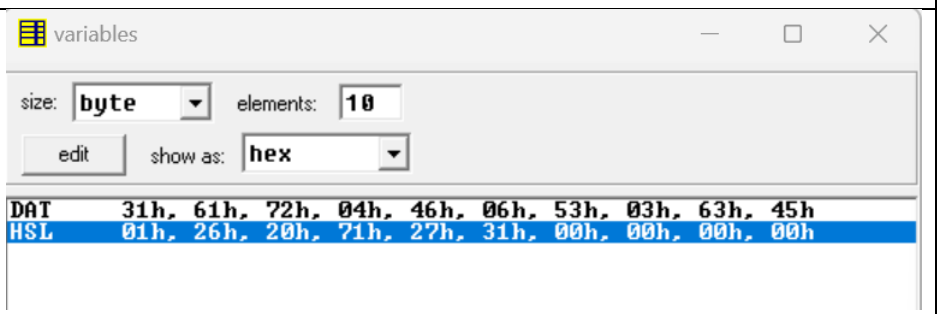
Program :

```
TITLE NO5 (OKTAL)
.MODEL SMALL
.STACK 100H
.DATA
DAT DB 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0
HSL DB 0,0,0,0,0,0
.CODE
MAIN PROC
     MOV AX, @DATA
     MOV DS, AX
     CMP CX, 0
     JZ SANA
     CMP BX, 0
     JZ SANA

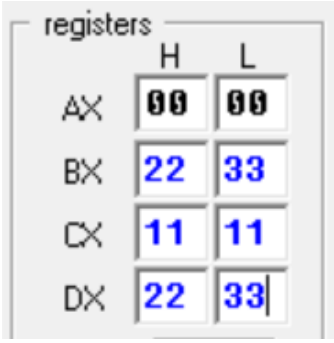
  SINI:  MOV AL, [DAT]
         ADD AL, BL
         CALL SDAA
         MOV [DAT], AL
         MOV AL, [DAT+1]
         ADC AL, BH
         CALL SDAA
         MOV [DAT+1], AL
         MOV AL, [DAT+2]
         ADC AL, CL
         CALL SDAA
         MOV [DAT+2], AL
         MOV AL, [DAT+3]
         ADC AL, 0
         CALL SDAA
         MOV [DAT+3], AL
         MOV AL, [DAT+15]
         ADD AL, 1
         CALL SDAA
         MOV [DAT+15], AL
         CMP [DAT+15],DL
         JNZ SINI
         ;PENJUMLAHAN SEBANYAK DH KALI
    SONO:MOV AL, [DAT+4]
         ADD AL, BL
         CALL SDAA
         MOV [DAT+4], AL
         MOV AL, [DAT+5]
         ADC AL, BH
         CALL SDAA
         MOV [DAT+5], AL
         MOV AL, [DAT+6]
         ADC AL, CL
         CALL SDAA
         MOV [DAT+6], AL
         MOV AL, [DAT+7]
         ADC AL, 0
         CALL SDAA
         MOV [DAT+7], AL
         MOV AL, [DAT+16]
         ADD AL, 1
         CALL SDAA
         MOV [DAT+16], AL
         CMP [DAT+16],DH
         JNZ SONO

    SUNU:MOV AL, [DAT+8]
         ADD AL, BL
         CALL SDAA
         MOV [DAT+8], AL
         MOV AL, [DAT+9]
         ADC AL, BH
         CALL SDAA
         MOV [DAT+9], AL
         MOV AL, [DAT+10]
         ADC AL, CL
         CALL SDAA
         MOV [DAT+10], AL
         MOV AL, [DAT+11]
         ADC AL, 0
         CALL SDAA
         MOV [DAT+11], AL
         MOV AL, [DAT+17]
         ADD AL, 1
         CALL SDAA
         MOV [DAT+17], AL
         CMP [DAT+17],CH
         JNZ SUNU
         MOV AL, [DAT+0]
         MOV [HSL+5], AL
         MOV AL, [DAT+1]
         ADC AL, [DAT+4]
         CALL SDAA
         MOV [HSL+4], AL
         MOV AL, [DAT+2]
         ADC AL, [DAT+5]
         CALL SDAA
         MOV AL, [DAT+5]
         ADD AL, [DAT+8]
         CALL SDAA
         MOV [HSL+3], AL
         MOV AL, [DAT+3]
         ADC AL, [DAT+12]
         CALL SDAA
         MOV [DAT+3], AL
         MOV AL, [DAT+13]
         ADC AL, 0
         CALL SDAA
         MOV [DAT+13], AL
         MOV AL, [DAT+3]
         ADD AL, [DAT+6]
         CALL SDAA
         MOV [DAT+6], AL
         MOV AL, [DAT+13]
         ADC AL, 0
         CALL SDAA
         MOV [DAT+13], AL
         MOV AL, [DAT+9]
         ADD AL, [DAT+6]
         CALL SDAA
         MOV [HSL+2], AL
         MOV AL, [DAT+7]
         ADC AL, [DAT+13]
         CALL SDAA
         MOV [DAT+7], AL
         MOV AL, [DAT+14]
         ADC AL, 0
         CALL SDAA
         MOV [DAT+14], AL
         MOV AL, [DAT+7]
         ADD AL, [DAT+10]
         CALL SDAA
         MOV [HSL+1], AL
         MOV AL, [DAT+11]
         ADC AL, [DAT+14]
         CALL SDAA
         MOV [DAT+11], AL
         MOV AL, [DAT+11]
         MOV [HSL], AL
SANA:MOV AH, 4CH
     INT 21H
MAIN ENDP
```

Hasil Program :

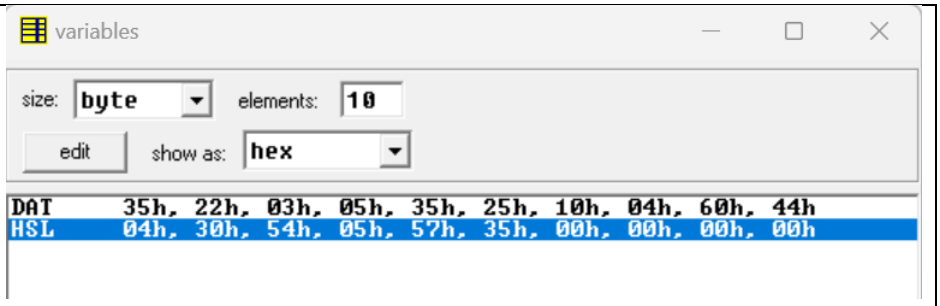
| Data Percobaan | Hasil Percobaan |
|---|---|
| 112233 * 112233 = 12620712731 |  |





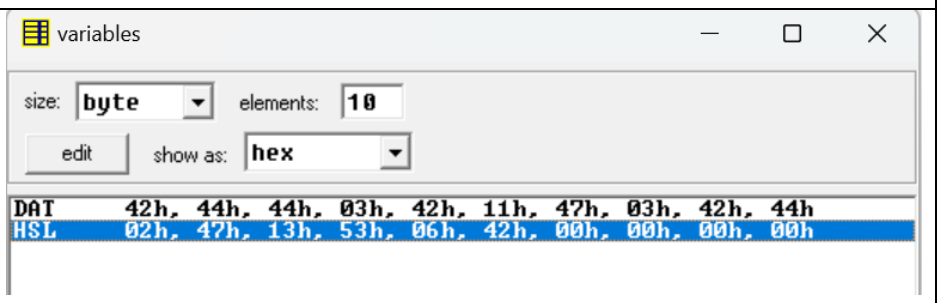
212223 * 201717= 43054055735



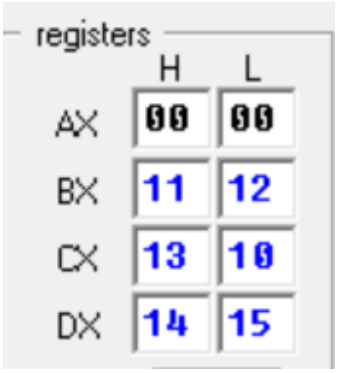



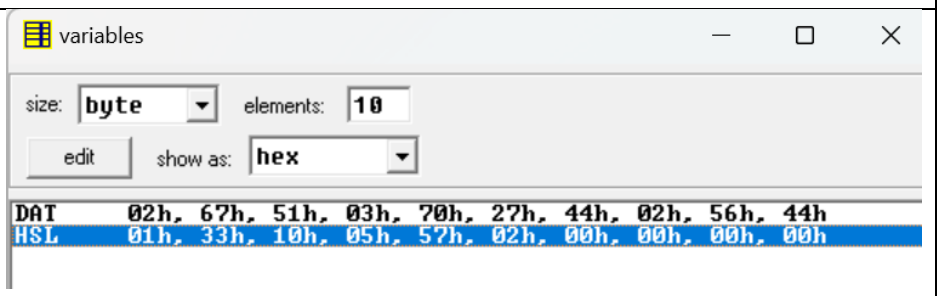

222222 * 111111 = 24713530642



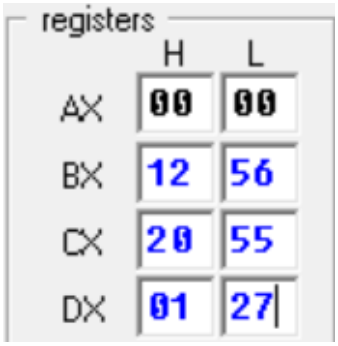



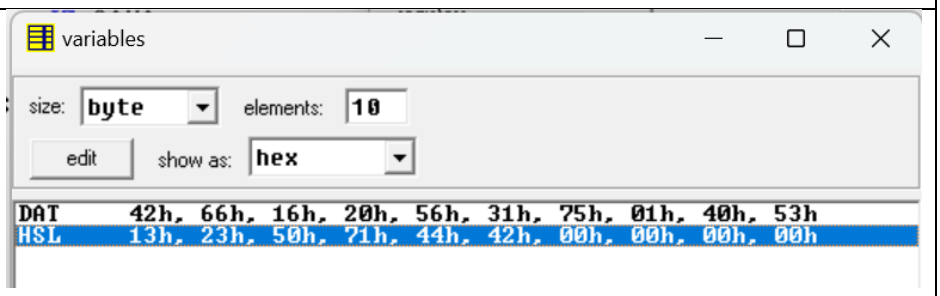

101112 * 131415 = 13310055702




551256 * 200127= 132350714442



Analisis Data :

Percobaan soal ke-4 merupakan program assebmly EMU8086 yang menangani operasi perkalian dalam sistem bilangan oktal (basis 8). Program ini mengimplementasikan algoritma perkalian dua bilangan oktal menggunakan metode penjumlahan berulang. Program ini menyiapkan data dalam array DAT dan menyimpan hasil dalam array HSL. Register-register penting seperti AX, BX, CX, dan DX digunakan sebagai tempat sementara (buffer) untuk data dan pengontrol loop. Data awal diinputkan pada register CL:BX untuk 6 digit awal dan pengalinya pada CH:DX.

Program operasi dimulai dari label SINI, yang menginisialisasi dan mengatur data awal dari array DAT. Proses utama perkalian dilakukan melalui pengulangan penambahan data sebanyak nilai dari bilangan pengali, yang disimpan di register DH. Setiap proses penambahan membutuhkan koreksi hasil agar tetap dalam format oktal yang benar, dan hal ini ditangani oleh pemanggilan subrutin SDAA.

Lopp SONO dan SUNU merupakan bagian dari pengulangan tambahan yang menangani digit yang lebih besar dalam array dan menyesuaikan carry yang terjadi selama proses penjumlahan. Instruksi ADD, ADC, dan CALL SDAA digunakan secara berulang dalam setiap blok untuk menjamin operasi aritmetika tetap konsisten dalam format oktal. Akhir dari proses ditandai dengan penyimpanan hasil akhir ke array.

## F. KESIMPULAN

Pada praktikum ini dapat disimpulkan bahwa praktikan dapat lebih memahami cara kerja software EMU8086 yang dapat menjalankan program yang digunakan untuk mikroprosesor. Praktikan juga lebih memahami bagaimana cara membuat program untuk mengalikan data pada bilangan oktal yang akan ditampilkan dalam bentuk BCD pada monitor. Operasi perkalian dilakukan menggunakan penggunaan metode penjumlahan berulang, sehingga perintah yang digunakan adalah perintah operasi penjumlahan pada operasi bilangan oktal. Penggunaan perintah DAA dan subrutin SDAA sangat membantu praktikan untuk menjalankan program penjumlahan dalam bentuk oktal mendapatkan nilai yang sesuai dengan perhitungan. Dan pada praktikum ini juga, praktikan mengetahui bagaimana hasil penjumlahan pada program dapat disimpan dan ditampilkan pada memory(vars) ataupun pada register juga di software EMU8086.

## DAFTAR PUSTAKA

Arief Wisaksono, I. M. (2019). *Buku Ajar Mikroprosesor* . Retrieved from eprint.umsida.ac.id:http://eprints.umsida.ac.id/7199/1/31.%20buku%20ajar%20mikroprosessor%20komplate%20-%20Arief%20Wisaksono.pdf

Hasanah, Rahmawati. 2019. Mikroprosesor dan Interface. Bandung: Politeknik Negeri Bandung

S.to. 2001. “Pemrograman Dengan Bahasa Assembly Edisi Online Versi 1.0